1. Source Code Implementation

1.1 Import required libraries

```
import cv2
import torch
import numpy as np
import pandas as pd
from ultralytics import YOLO
import os
```

1.2 Dataset configuration using data.yaml

Before training, we define the dataset configuration. The dataset consists of 7,200 images with 54,815 annotated honey bee bounding boxes.

```
# Create data.yaml for YOLO11 training
dataset_yaml = """
path: ./dataset
train: images/train  # 4,880 images
val: images/val      # 800 images
test: images/test    # 1,520 images

nc: 1
names: ['honey_bee'] # Class 0
"""

# Save configuration to a file
with open('data.yaml', 'w') as f:
    f.write(dataset_yaml)
```

1.3 Load the pretrained YOLO11n model

```
# Initialize YOLO11 nano model with pretrained weights
model = YOLO('yolo11n.pt')
```

1.4 Configure GPU/CUDA device

```
# Check for CUDA availability and configure the device
# Target GPU: NVIDIA GeForce RTX 5050 Laptop GPU 8GB
device = 'cuda' if torch.cuda.is_available() else 'cpu'
print(f"Computation device configured: {device}")
```

1.5 Train the YOLO11n model using the prepared honey bee dataset

```
# Train the model with specific project parameters
# Note: Actual training stopped at 50 epochs due to early stopping
```

```
results = model.train(
    data='data.yaml',
    epochs=100,           # Maximum epochs: 100
    imgsz=640,            # Image size for training
    batch=13,             # Batch size
    patience=20,          # Early stopping patience
    device=device,
    project='honey_bee_detection',
    name='yolo11n_training'
)
```

1.6 Validate the trained model

```
# Evaluate model performance on the validation set (800 images)
# Expected results: Precision: 94.50%, Recall: 93.10%, F1: 93.80%
# mAP@50: 95.00%, mAP@50-95: 78.50%
metrics = model.val(data='data.yaml', device=device)
```

1.7 Load the best trained weights (best.pt)

```
# Load the optimal weights obtained after training
best_model_path = 'honey_bee_detection/yolo11n_training/weights/best.pt
best_model = YOLO(best_model_path)
```

1.8 Perform honey bee detection on a test image & 1.11 Apply Non-Maximum Suppression

```
# 1.8 Detection on a test image (1920x1080)
# 1.11 NMS is applied internally during inference via conf and iou thre
test_image_path = 'dataset/images/test/sample_test.jpg'

image_results = best_model.predict(
    source=test_image_path,
    conf=0.25,  # Confidence threshold
    iou=0.70,   # IoU threshold for Non-Maximum Suppression
    device=device
)
```

1.9 Draw bounding boxes and confidence scores & 1.10 Count the detected honey bees

```
for r in image_results:
    # 1.9 plot() automatically draws bounding boxes, labels, and conf s
    annotated_img = r.plot()

    # 1.10 Count the detected honey bees
    bee_count = len(r.boxes)
    print(f"Detected {bee_count} honey bees in the test image.")

    # Display the output test image
```

```
    cv2.imshow("Detected Honey Bees - Test Image", annotated_img)
    cv2.waitKey(0)
    cv2.destroyAllWindows()
```

1.12 Perform honey bee detection on video

```
video_path = 'Honeybees.webm'
output_video_path = 'Honeybees_yolo11n_annotated.mp4'
csv_output_path = 'Honeybees_frame_counts.csv'

# Initialize video capture (Target: 1920x1080, 29.97 FPS, 3676 frames)
cap = cv2.VideoCapture(video_path)
width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps = cap.get(cv2.CAP_PROP_FPS)

# Define codec and create VideoWriter object for the annotated output
fourcc = cv2.VideoWriter_fourcc(*'mp4v')
out = cv2.VideoWriter(output_video_path, fourcc, fps, (width, height))

frame_counts = []
frame_number = 0

print("Starting video inference...")
while cap.isOpened():
    ret, frame = cap.read()
    if not ret:
        break

    frame_number += 1

    # Run YOLO11n inference on the current frame
    results = best_model.predict(
        source=frame,
        conf=0.25,
        iou=0.70,
        device=device,
        verbose=False # Suppress per-frame terminal output for cleaner
    )

    r = results[0]

    # Draw boxes and scores on the frame
    annotated_frame = r.plot()

    # 1.13 Calculate the number of detected bees in every video frame
    num_bees = len(r.boxes)
    frame_counts.append({
        'Frame': frame_number,
        'Bee_Count': num_bees
    })
```

```
    # Write the frame into the output video file
    out.write(annotated_frame)

# Release video objects
cap.release()
out.release()
```

1.14 Calculate minimum, maximum, and average bee count

```
# Convert frame counting data to a pandas DataFrame
df_counts = pd.DataFrame(frame_counts)

# Compute video statistics
min_bees = df_counts['Bee_Count'].min()
max_bees = df_counts['Bee_Count'].max()
avg_bees = df_counts['Bee_Count'].mean()

print("\n--- Video Inference Statistics ---")
print(f"Total Frames Processed: {frame_number} (~122.66 seconds)")
print(f"Minimum Detected Bees: {min_bees}")
print(f"Maximum Detected Bees: {max_bees}")
print(f"Average Detected Bees per Frame: {avg_bees:.2f}")
```

1.15 Save the annotated output video & 1.16 Save frame-wise bee counts into a CSV file

```
# 1.15 Output video is already saved sequentially during the loop above
print(f"Annotated output video saved successfully to: {output_video_pat

# 1.16 Save frame-wise bee counts into a CSV file
df_counts.to_csv(csv_output_path, index=False)
print(f"Frame-wise count data saved successfully to: {csv_output_path}"
```